

Thakaa at AraGenre 2026: Definition-Guided LLM Judging with Full-Context Multi-Model Consensus for Hierarchical Arabic Genre Classification

Hassan Alqaeri

Thakaa, Advanced AI &
Information Technology

h.alqaeril@thakaa.it.net

Meshal Alamr

Thakaa, Advanced AI &
Information Technology

m.alamr@thakaa.it.net

Abstract

We describe the winning system for the AraGenre 2026 shared task on hierarchical, definition-guided Arabic genre classification, where each of 27,972 hidden-test Arabic segments must be assigned one of 6 broad genres and one of 74 fine-grained specific genres whose English definitions are provided but whose specific-label inventory is *disjoint* from the released training data, the 6 broad genres being shared. Our approach uses no gradient-based training on the task, though the judge prompt does carry hand-written per-genre cues (§4). A family-restricted, definition-guided LLM judge performs setwise scoring of candidate genres within a hierarchically constrained decoding scheme, reaching 0.7139 Hierarchical Macro F1. From the public scoreboard we diagnose that our system already led on Specific Macro F1 but trailed on Broad Macro F1, and we narrow that gap with a full-context multi-model consensus correction stage: two instruction models are each shown the *entire* 74-definition taxonomy and re-predict the specific genre; corrections are applied only where both models agree against the base system. Feeding the full taxonomy almost halves the residual topic-trap errors, where a book *about* religion is taken for religious scripture. A final attractor-drain step redistributes over-predicted generic classes to starved rare classes. The system scores 0.7352 Hierarchical Macro F1, ranking 1st, +0.018 over the next team and improving every metric over our base.

1 Introduction

Genre is what a text *does* rather than what it is about: the same subject appears in a contract, a news report and a textbook, and only the first is useful to someone assembling a legal corpus. Sorting documents this way is what makes an archive filterable and a corpus assemblable by register, and Arabic has far less of this work behind it than English. AraGenre 2026 (El-Haj et al., 2026) frames

the problem as one of generalisation: systems receive English genre definitions and limited synthetic training data, but the hidden benchmark contains noisier naturally occurring Arabic spanning Modern Standard Arabic, Classical Arabic, and multiple dialects, with *previously unseen genre-definition combinations*. Because the 74 specific test genres are disjoint from training, memorisation is useless and semantic understanding of communicative function is required. The official ranking metric is

$$\text{Hier. F1} = \frac{1}{2} (\text{Broad F1}_{\text{macro}} + \text{Spec. F1}_{\text{macro}}).$$

Our contributions are:

1. A hierarchically constrained, definition-guided LLM judge that reaches 0.7139 Hierarchical Macro F1 with no task training.
2. A scoreboard-driven diagnosis showing the two tiers must be optimised differently, and that our deficit against the leading system sat entirely in the broad tier.
3. A full-context multi-model consensus correction, applied conservatively (agreement-gated, direction-verified), with a three-arm ablation separating the effect of the taxonomy from that of an explicit type-vs-topic instruction.
4. An attractor-drain step targeting the macro-F1 failure mode of over-predicting generic classes.
5. Empirical validation on the hidden test: 0.7352 Hierarchical Macro F1, ranked 1st of 18 teams, each component scored as its own submission.

Code and prompts: <https://github.com/Hassn11q/thakaa-aragenre-2026>. The main challenges were a specific-label inventory disjoint from training, a topic trap (writing *about* a

subject vs. the subject’s genre), and classes with no written signal (song dialects, textbook levels).

2 Related Work

Arabic and hierarchy-aware HTC. Arabic pre-trained models (Antoun et al., 2020, 2021) underpin most classification work on the language, including hierarchical variants (Bouchiha et al., 2025, 2026), and a large literature models the label hierarchy explicitly (Zhu et al., 2023; Cai et al., 2024; Zeng et al., 2025; Wang et al., 2025; du Toit and Dunaiski, 2025; Zangari et al., 2024). These closed-label supervised setups do not transfer to AraGenre’s unseen, definition-specified labels; we instead enforce the hierarchy structurally, forcing broad = parent(specific).

Zero-shot HTC, LLM judging, and consensus. Prior systems reach an unseen label space through prompt ensembles (Xia et al., 2025), LLM-built prototypes (Zhang et al., 2025) or rewritten taxonomies (Golde et al., 2026); we keep the definitions verbatim and vary only how much of the taxonomy is visible at once. Our two-stage design retrieves candidate definitions (Chen et al., 2024) and compares them with an LLM (Sun et al., 2023; Zheng et al., 2023), with Qwen3-Embedding (Qwen Team, 2025) bridging Arabic and English; the chain-of-thought pass follows reasoning-based HTC (Jiang et al., 2025; Pan and Wu, 2025). Ensembling diverse predictors (Dietterich, 2000) appears in LLM form as juries (Verga et al., 2024), blending (Jiang et al., 2023), mixtures of agents (Wang et al., 2024), debate (Du et al., 2024) and self-consistency (Wang et al., 2023); those aggregate free-form text or resample one model, whereas we require two models to emit the *same* label, which helps only with the full taxonomy in context.

3 Background

Each instance is {id, text}; systems output one `broad_genre` (of 6: Informative, Creative, Interactive, Learning, Legal, Religious) and one `specific_genre` (of 74), using exact taxonomy label names, each with an English definition. The released data are tiny and synthetic-like (train: 140 items / 7 specific genres; dev: 110 / 6), and the specific-label inventories are 100% disjoint across train, dev and test, so no supervised label set transfers. The *hidden test* has 27,972 naturally occurring instances over 74 specific genres in 6 fami-

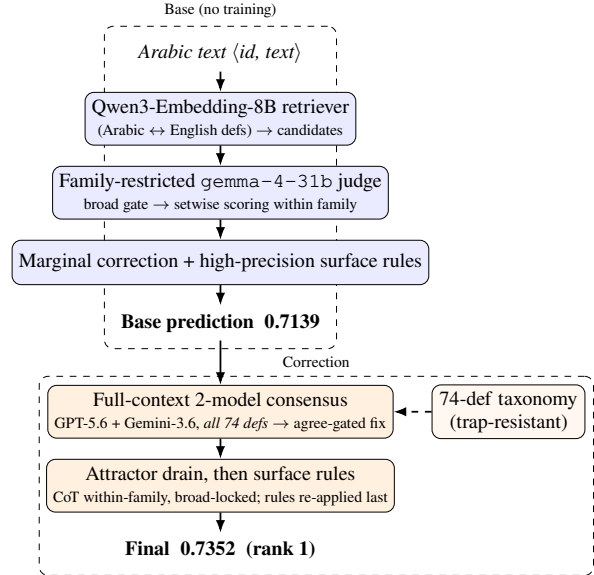


Figure 1: System overview. The base judge yields 0.7139; full-context two-model consensus plus a within-family attractor drain correct both tiers, reaching 0.7352 (rank 1).

lies: Informative (42), Learning (12), Creative (7), Religious (6), Interactive (4) and Legal (3). Informative is dominated by 20 job domains and 13 book-description topics; Creative holds 5 song dialects. Missing predictions are counted incorrect; the stronger official dev baseline is in Table 1. Informative dominates our predictions (~50%), and the specific tier’s 74 classes make Macro F1 highly sensitive to rare-class recall (e.g., dialect song lyrics, niche job domains).

4 System

4.1 Base: Family-Restricted Definition-Guided Judge

Figure 1 summarises the pipeline. We serve google/gemma-4-31B-it (Gemma Team, Google DeepMind, 2026) through vLLM under the local endpoint alias gemma-4-31b; it acts as a setwise judge. The API model strings are pinned in Appendix G. For each text: (i) Qwen3-Embedding-8B (Qwen Team, 2025) encodes the Arabic text and the English candidate definitions and proposes candidate specific genres by cosine similarity, bridging Arabic text and English definitions; (ii) decoding is hierarchically constrained: the broad genre is gated first and specific candidates are restricted to that broad family (all siblings for small families, top-*k* retrieved for large topical ones), so that

broad = parent(specific) and broad *accuracy* equals the family-accuracy of the specific prediction (Broad Macro F1 does not: it averages per-class F1, and the gap between the two is what §4.1 exploits); (iii) the judge scores each candidate 0–100 against its definition in one setwise call (Sun et al., 2023) and we take the argmax after a light marginal correction (Appendix F); and (iv) a few high-precision surface rules override the judge only on near-unambiguous markers (Qur’anic mushaf orthography → quran; isnād openings → hadith; we screened samples by hand and saw no false positives, though without gold this is an impression, not a precision estimate; emoji/hashtag short posts → Interactive subtypes). The judge also sees 74 hand-written per-genre contrastive discriminators appended to the definitions; hyperparameters are in Appendix G. On the hidden test this base pipeline scores 0.7139.

Diagnosis from the scoreboard. The leaderboard exposed the structure (Table 2): our base led on Specific Macro F1 (0.5487 vs. 0.5339) but trailed on Broad Macro F1 (0.8792 vs. 0.90), so the whole deficit sat in the broad tier. Broad accuracy (0.9003) exceeded broad macro, indicating uneven per-class performance; reading the errors showed cross-family false positives, with Bible passages filed as poetry or encyclopedia entries and news filed as legal.

4.2 Full-Context Multi-Model Consensus Correction

We re-predict the specific genre with two frontier instruction models (GPT-5.6 Luna (OpenAI, 2026) and Gemini-3.6 Flash (Google, 2026)), *feeding each the entire 74-definition taxonomy* ($\approx 2.9\text{K}$ tokens) grouped by broad genre, and asking for the single best specific label. This follows the ensemble argument that combining diverse predictors beats any single one (Dietterich, 2000), in its LLM form (Verga et al., 2024; Wang et al., 2024); we claim model diversity, not statistical independence. (Claude Sonnet 5 (Anthropic, 2026) was run on the diagnostic pool but excluded from the deployed consensus on cost grounds.)

What suppresses the topic trap. Given only the 6 broad definitions, all three verification models fell into a *topic trap* on the book descriptions: text *about* Islam → Religious, even when it was an Informative *book description*. The full-taxonomy prompt also carries an explicit type-vs-topic in-

struction, so we ran a three-arm ablation over the same 1,403 instances to estimate each factor’s incremental effect (one model, GPT-5.6, identical pool): with broad definitions only and no such instruction, 394 are called Religious; adding the instruction alone removes 41% of those (394→233); supplying the full taxonomy removes a further 49% (233→119). Both factors matter and neither explains the effect alone: the instruction states the convention, the taxonomy shows the label that realises it. Scripture filed as an encyclopedia entry (هؤلاء امراء بني عيسو, Genesis 36) is moved to bible, and base-family agreement rises in five of the six families (Appendix D).

Conservative application. The first model runs over all 27,972 instances, the second only over the 8,041 where the first disagrees with the base, and a correction is applied only where both name the same label. Broad-changing moves are gated to seven read-verified directions, led by Informative→Interactive for comments and Legal→Informative for wire news (Appendix H); unreliable directions (e.g. Informative→Learning, which over-fires student-writing on academic prose) are rejected, 718 moves in all. Within-family refinements are applied wherever both models agree.

4.3 Attractor Drain (Specific Macro)

Reading our own hidden-test predictions during the evaluation phase revealed five large “attractor” classes that absorb texts the judge cannot place, starving rare siblings. They come from that prediction distribution, not from held-out gold. The drain is deterministic: when the base label is an attractor class and a separate chain-of-thought re-judging pass (Wei et al., 2022) names a different sibling in the same family, the sibling replaces it (broad never changes). The surface rules are then re-applied over the corrected labels, acting as a guard rather than a gain (Appendix H). On the base alone it moves 554 instances, 6–35% of each attractor class (Appendix E); in the final pipeline consensus runs first, leaving the drain 383. Relative to the base, the final system changes 459 broad and 2,890 specific labels (10.3% of instances).

5 Experimental Setup

Data splits. We train nothing; every system is applied unchanged to the hidden test. An input is {`"id"`, `"text"`} with Arabic text such as

System	Split	Hier. F1
Official baseline (embedding sim.)	dev	0.4658
Our judge (gemma)	dev	0.8794
Our judge (GPT-5.6)	dev	0.8943
gemma + consensus	dev	0.8557
GPT-5.6 judge + attractor drain [†]	dev	0.9358
Genre-general zero-shot judge	test	0.6067
+ embedding retrieval gate	test	0.6812
family-restricted, derived broad	test	0.6882
+ rules / calibration (base)	test	0.7139
+ 2-model consensus (pool)	test	0.7224
+ safe broad expansion (full set)	test	0.7256
+ full consensus + drain (final)	test	0.7352

Table 1: Main progression on both splits. Dev is near-degenerate (its 6 specific genres map one-to-one onto 6 broad, so all three F1 metrics coincide), so the hidden test is our primary evaluation. [†]applied to the GPT-5.6 judge, tuned on dev, so not held out.

مطلوب سائق باص 24 راكب في شركة في دبي جبل علي (test id 24486, “bus driver wanted, 24-seater, for a company in Dubai, Jebel Ali”); the required output pairs it with `broad_genre = Informative` and `specific_genre = drivers_and_delivery_jobs`.

Preprocessing. None. Normalisation erases the two orthographic signals that separate whole families here: heavy diacritisation marks 77% of the texts we label quran and 43% of poetry against 1% elsewhere, and emoji mark 47% of Interactive posts against under 1% (Appendix G).

Tools. `transformers`, `sentence-transformers` and `bitsandbytes` for Qwen3-Embedding-8B, vLLM for the local judge, and the `openai` and `google-genai` SDKs for the verifiers; versions and URLs in Appendix G.

Metrics. Macro F1, Weighted F1 and Accuracy at both tiers; the official ranking metric is Hierarchical Macro F1, the mean of Broad and Specific Macro F1.

Cost. Consensus makes 36,013 calls (27,972 + 8,041) with a $\approx 3.0\text{K}$ -token prompt each, about US\$50 in total; rates and timings are in Appendix G. The base pipeline needs no paid API.

6 Results

6.1 Development-Set Results and Component Transfer

Re-running the pipeline on dev (110 examples, official gold) separates what transfers from what does

System	Hier	SpM	SpW	SpA	BrM	BrW	BrA
Best rival [‡]	.717	.534	.581	.599	.900	.912	.911
Base (ours)	.714	.549	.584	.591	.879	.900	.900
+ consensus	.722	.553	.592	.598	.892	.910	.910
Final (rank 1)	.735	.573	.613	.619	.898	.914	.914

Table 2: Official metrics on the hidden test, rounded to three decimals (Sp/Br = Specific/Broad; M/W/A = Macro F1, Weighted F1, Accuracy). We beat the second-placed system ([‡], submission 870633) on six of seven; the exception is Broad Macro F1 (.898 vs. .900), and we lead Specific Macro F1 by +.039.

not. The drain transfers: no class list is carried over from the test set, and the two dev classes it fires on are the two most over-predicted there ($1.30\times$ and $1.25\times$ gold frequency). It moves five labels for +0.042 (paired bootstrap over the 110 dev items, 10k resamples: 95% CI [+0.010, +0.082]). On the hidden test, by contrast, the five attractors were picked by reading (§4). The consensus stage does not: dev has no `*_book_description` and no scripture, so the trap it repairs has zero instances; two corrections fire and both are wrong. Residual dev errors stem from a convention clash between splits (Appendix A).

7 Analysis

Broad Macro is dominated by minority-class false positives, Specific Macro by rare-class recall, so gains on both compound through the mean. Broad-only prompting produced 157 topic-trap errors on the book descriptions that all three models made unanimously, which the full taxonomy largely removed; and a single model disagreed with the base on $\sim 29\%$ of specifics, too noisy to act on without a second. Every alternative we tried for the correction stage fell below the 0.7139 base; the scores are in Appendix C.

Error analysis (hidden test). Three types dominate: cross-family topic leaks, attractor over-prediction, and ceiling classes where 87–99% of the texts we label as song lyrics carry no marker from that dialect’s cue list. Examples in Appendix B.

8 Conclusion

Training-free hierarchical Arabic genre classification is achievable: a constrained definition-guided judge, an agreement-gated consensus and a rare-class attractor drain ranked 1st of 18 at 0.7352 Hierarchical Macro F1. The taxonomy matters because

it gives the model a label to move an error *to*; song-lyric dialect remains the clearest ceiling.

Acknowledgements

We thank the AraGenre 2026 organisers for designing and running the shared task, for the released taxonomy and definitions, and for their prompt support throughout the evaluation phase. We also thank the maintainers of the open models and libraries our pipeline builds on. Our code, prompts, cached model outputs and submission builder are available at <https://github.com/Hassn11q/thakaa-aragenre-2026>.

Limitations

Every hidden-test *score* we report is an official evaluation-phase submission (Appendix H); we made no submission after the deadline. Two kinds of number are not submissions and should not be read as such. The instruction-only arm of the three-arm ablation (§4) was run after the evaluation phase closed, so that comparison is post hoc; and the descriptive tables in Appendices B, D, E and H are analyses of the submitted outputs, computed locally rather than scored. The hidden test has no released gold labels and the development set is near-degenerate, so component decisions were validated on the public leaderboard and by manual reading rather than on held-out gold. Our reported progression therefore doubles as the selection signal: successive submissions were guided by leaderboard feedback, which risks fitting to the test distribution. The one component we can test against gold is the drain, on dev, where a paired bootstrap puts its gain above zero (95% CI [+0.010, +0.082]). Every hidden-test gain is without an interval because that split has no released gold, so those gains should be read as leaderboard-measured rather than statistically certified, and we can offer no uncertainty estimate on the winning margin. Consensus correction incurs API cost proportional to test size. Song-dialect distinctions remain near the noise floor because the distinguishing features are phonetic and largely absent from written lyrics.

Reproducibility Statement

Official submission. The results we report are those of submission 872515, evaluated 1 August 2026, which scored 0.7352 Hierarchical Macro F1 (Specific Macro 0.5725, Weighted 0.6125, Accuracy 0.6189; Broad Macro 0.8979, Weighted

0.9142, Accuracy 0.9138) over 27,972 of 27,972 gold instances with no missing prediction, ranking 1st of the 18 teams in the organisers’ final standings, which we transcribe in the repository for reference. Every hidden-test number in Table 1 and Appendix C is likewise an official scored submission rather than a local estimate; the submission IDs are listed in Appendix H. All inference is zero-shot with no task-specific training.

Data: we use only the official AraGenre inputs and definition files, and no external labelled data. The consensus verifiers receive the 74 definitions verbatim. The judge does not: by default it appends a one-line Arabic discriminator to each definition and prepends per-family cue lists, both author-written from the released definitions and Arabic linguistic knowledge, and both naming genres explicitly (`src/discriminators.py`, `src/judge.py`). The surface rules name a further seven labels. No test label or annotation informs any of it, but the pipeline is not label-agnostic and we do not claim it is.

Models: the base retriever is Qwen3-Embedding-8B (4-bit NF4 weights with fp16 compute, 4096-dim, cosine over L2-normalised vectors, sequence length capped at 512 tokens); the judge is a Gemma-family model (`gemma-4-31b`) served via vLLM with setwise 0–100 scoring; the consensus verifiers are GPT-5.6 Luna and Gemini-3.6 Flash, each queried with the full-taxonomy prompt.

Settings: the judge decodes at temperature 0 with seed 42; the Gemini verifier runs at temperature 0 and the GPT verifier at `reasoning_effort=low` with no temperature set. The retriever caches the top 20 specific genres per text; the judge scores every sibling in the four small families and the top 6 retrieved elsewhere, in a single setwise pass, with marginal correction $\alpha=0.5$.

Pipeline and code: paths below are relative to the repository root. Every stage is released, one file each: retrieval (`src/retrieve.py`), the family-restricted judge (`src/judge.py`), the surface rules (`src/rules.py`), the full-context verifiers (`src/verify.py`), the consensus and drain assembly (`src/assemble.py`), and the submission builder (`src/submission.py`), which enforces `broad=parent(specific)` and validates that all 27,972 IDs are present with exact labels.

Two cached inputs. The broad gate the judge re-

stricts to is submission 866538 (0.6812), an earlier retrieval-gated system we submitted and scored, reused verbatim rather than regenerated by the released code; and the base predictions additionally carry an Interactive-recall reassignment applied during the evaluation phase by a separate automatic judge pass (`src/interactive_judge.py`), which we release but which is not wired into `reproduce.sh`; re-running the released judge over the same candidate sets reproduces 93.5% of the base specific labels and 97.1% of the broad labels. Both files ship in `artifacts/`, so the submitted predictions rebuild exactly (27,972/27,972) from cached outputs, but the base stage is a cached input rather than something the repository regenerates end to end.

Caveats: the hosted API models are versioned by release date and are not guaranteed bit-reproducible across dates; the consensus stage incurs API cost proportional to the number of verified instances (on the order of tens of US dollars for this test set). The base pipeline runs on a single CUDA GPU (embeddings) plus a remote vLLM endpoint (judge).

Ethics Statement

This work uses only the publicly released Ara-Genre shared-task data and introduces no new data collection; we did not gather, store, or infer personal information, and any user-generated text (e.g., social-media posts) originates from the official benchmark. In line with the shared-task rules, all predictions are produced automatically by the system: no submitted label was hand-edited and we did not reverse-engineer the hidden evaluation labels. We did read model outputs and the corresponding texts to design automatic correction rules, and the rule-precision estimates in §4 come from that inspection; this is author inspection of system outputs, not annotation of the evaluation set, and no such judgement enters a submission except through a deterministic rule.

Genre classification is a comparatively low-risk task, but two considerations warrant care. First, the taxonomy spans religiously and culturally sensitive categories (Islamic and Christian scripture, dialectal varieties, regional song lyrics), and we treat these descriptively as text-type/register labels; misclassification carries no endorsement or value judgement, and the system is not intended to assess the authenticity, orthodoxy, or quality of re-

ligious or cultural texts. Second, dialect-related labels can encode regional and social signals; models trained on skewed corpora may under-serve lower-resourced dialects (e.g., the near-noise-floor separation of Sudanese and Iraqi song lyrics we report), which practitioners should weigh before deployment.

Potential misuse. Two uses of this system would concern us. The taxonomy separates song lyrics by regional dialect, so the same classifier can be pointed at ordinary prose and read as a guess at where a writer is from; dialect is not provenance, our own measurements show the written signal is mostly absent, and we would not treat the output as an author attribute. And because the labels distinguish religious from informative text, genre output could be used to route or suppress material by register rather than by content. The system assigns text types, it does not assess them, and a 0.57 Specific Macro F1 is not a basis for an automated decision about a document or its author without a person in the loop.

Our correction stage sends Arabic text to third-party hosted LLM APIs; because the inputs are already-public shared-task data, this poses no additional privacy exposure, but the same pipeline should not be applied to private or sensitive text without review. The approach relies on large hosted models whose availability, cost, and behaviour may change, which limits equitable reproducibility; the base retriever-plus-open-judge pipeline is provided as a lower-cost, self-hostable alternative.

References

- Ilias Aarab. 2026. BTZSC: A benchmark for zero-shot text classification across cross-encoders, embedding models, rerankers and LLMs. *arXiv preprint arXiv:2603.11991*.
- Muhammad Abdul-Mageed, Chiyu Zhang, Abdel-Rahim Elmadany, Houda Bouamor, and Nizar Habash. 2021. NADI 2021: The second nuanced Arabic dialect identification shared task. In *Proceedings of the Sixth Arabic Natural Language Processing Workshop*.
- Anthropic. 2026. Claude Sonnet 5. Model documentation, <https://www.anthropic.com/news/claude-sonnet-5>.
- Wissam Antoun, Fady Baly, and Hazem Hajj. 2020.

- AraBERT: Transformer-based model for Arabic language understanding. In *Proceedings of the 4th Workshop on Open-Source Arabic Corpora and Processing Tools*.
- Wissam Antoun, Fady Baly, and Hazem Hajj. 2021. AraGPT2: Pre-trained transformer for Arabic language generation. In *Proceedings of the Sixth Arabic Natural Language Processing Workshop*.
- Djelloul Bouchiha, Abdelghani Bouziane, Nouredine Doumi, and Benamar Hamzaoui. 2025. [Fine-tuning AraGPT2 for hierarchical Arabic text classification](#). *Science, Engineering and Technology*, 5(1):55–65.
- Djelloul Bouchiha, Benamar Hamzaoui, Abdelghani Bouziane, Nouredine Doumi, Farouk Omar Berbouchi, Aymen Abdelghani Kebir, Nihad Mebarki, and Badiâ Achouak Benameur. 2026. [Hierarchical Arabic text classification: Deep learning-based approach](#). *Annals of West University of Timișoara, Mathematics and Computer Science*, 62:1–15.
- Fuhan Cai, Duo Liu, Zhongqiang Zhang, Ge Liu, Xiaozhe Yang, and Xiangzhong Fang. 2024. NER-guided comprehensive hierarchy-aware prompt tuning for hierarchical text classification. In *Proceedings of LREC-COLING 2024*, pages 12117–12126.
- Huiyao Chen, Yu Zhao, Zulong Chen, Mengjia Wang, Liangyue Li, Meishan Zhang, and Min Zhang. 2024. [Retrieval-style in-context learning for few-shot hierarchical text classification](#). *Transactions of the Association for Computational Linguistics*, 12:1214–1231.
- Thomas G. Dietterich. 2000. Ensemble methods in machine learning. In *Multiple Classifier Systems (MCS)*, LNCS 1857, pages 1–15.
- Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2024. Improving factuality and reasoning in language models through multiagent debate. In *Proceedings of ICML*.
- Jaco du Toit and Marcel Dunaiski. 2025. Combining language and topic models for hierarchical text classification. *arXiv preprint arXiv:2507.16490*.
- Mo El-Haj, Saad Ezzini, Shadi Abudalfa, Salima Lamsiyah, and Mustafa Jarrar. 2026. AraGenre 2026: A hierarchical definition-guided Arabic genre classification shared task. In *Proceedings of the 4th Arabic Natural Language Processing Conference (ArabicNLP 2026)*, Budapest, Hungary. Association for Computational Linguistics.
- Gemma Team, Google DeepMind. 2026. [Gemma 4 technical report](#). Technical report, Google DeepMind. ArXiv:2607.02770.
- Jonas Golde, Nicolaas Paul Jedema, RaviKiran Krishnan, and Phong Le. 2026. Hierarchical text classification with LLM-refined taxonomies. In *Proceedings of the 19th Conference of the European Chapter of the Association for Computational Linguistics (EACL), Long Papers*, pages 214–228.
- Google. 2026. Gemini 3.6 flash. Model documentation, <https://ai.google.dev/gemini-api/docs/models/gemini-3.6-flash>.
- Dongfu Jiang, Xiang Ren, and Bill Yuchen Lin. 2023. LLM-blender: Ensembling large language models with pairwise ranking and generative fusion. In *Proceedings of ACL*.
- Lekang Jiang, Wenjun Sun, and Stephan Goetz. 2025. Reasoning for hierarchical text classification: The case of patents. *arXiv preprint arXiv:2510.07167*.
- OpenAI. 2026. GPT-5.6 luna. Model documentation, <https://developers.openai.com/api/docs/models/gpt-5.6-luna>.
- Shuaidong Pan and Di Wu. 2025. [Hierarchical text classification with LLMs via BERT-based semantic modeling and consistency regularization](#). *Preprints.org*.
- Qwen Team. 2025. Qwen3 embedding: Advancing text embedding and reranking through foundation models. *arXiv preprint arXiv:2506.05176*.
- Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhumin Chen, Dawei Yin, and Zhaochun Ren. 2023. Is ChatGPT good at search? investigating large language models as re-ranking agents. In *Proceedings of EMNLP*.

- Pat Verga, Sebastian Hofstatter, Sophia Althammer, Yixuan Su, Aleksandra Piktus, Arkady Arkhangorodsky, Minjie Xu, Naomi White, and Patrick Lewis. 2024. Replacing judges with juries: Evaluating LLM generations with a panel of diverse models. *arXiv preprint arXiv:2404.18796*.
- Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Zou. 2024. Mixture-of-agents enhances large language model capabilities. *arXiv preprint arXiv:2406.04692*.
- Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. 2023. Self-consistency improves chain of thought reasoning in language models. In *Proceedings of ICLR*.
- Yihong Wang, Zhonglin Jiang, Ningyuan Xi, Yue Zhao, Qingqing Gu, Xiyuan Chen, Hao Wu, Sheng Xu, Hange Zhou, Yong Chen, and Luo Ji. 2025. Making language model a hierarchical classifier. *arXiv preprint arXiv:2507.12930*.
- Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Brian Ichter, Fei Xia, Ed Chi, Quoc Le, and Denny Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems (NeurIPS)*.
- Mingxuan Xia, Zhijie Jiang, Haobo Wang, Junbo Zhao, Tianlei Hu, and Gang Chen. 2025. Ensembling prompting strategies for zero-shot hierarchical text classification with large language models. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 18189–18208.
- Alessandro Zangari, Matteo Marcuzzo, Michele Schiavinato, Lorenzo Giudice, Andrea Albarelli, Andrea Gasparetto, and Matteo Rizzo. 2024. Hierarchical text classification and its foundations: A review of current research. *Electronics*, 13(7):1199.
- Biqing Zeng et al. 2025. CPM-based hierarchical text classification. *Journal of Artificial Intelligence Research*, 82:367–388.
- Qian Zhang, Qinliang Su, Wei Zhu, and Yachun Pang. 2025. HierPrompt: Zero-shot hierarchical text classification with LLM-enhanced prototypes. In *Findings of the Association for Computational Linguistics: EMNLP 2025*, pages 3846–3859.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhonghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-judge with MT-bench and chatbot arena. In *NeurIPS*.
- He Zhu, Chong Zhang, Junjie Huang, Junran Wu, and Ke Xu. 2023. HiTIN: Hierarchy-aware tree isomorphism network for hierarchical text classification. In *Proceedings of ACL*.

A Annotation-Convention Divergence Between Splits

Five of the seven residual dev errors are texts that analyse religion from the outside, e.g. نشير بعض الدراسات إلى أن أساليب الوعظ تختلف بين المجتمعات (“some studies indicate that preaching styles differ between societies”). Our system labels these Analytical Reports (Informative); the dev gold label is Religious Commentary (Religious).

This is the *inverse* of the convention in the hidden test, where `islamic_book_description` (“texts summarising Islamic or religious books”) is Informative and our consensus stage is rewarded for moving such texts out of Religious. Both conventions are stated in their own definition files, and the two files disagree about where writing *about* religion belongs. A system that follows the test convention is therefore necessarily penalised on dev, which bounds how far dev accuracy can be pushed without split-specific rules. It also supports our central claim: the convention is carried by the definitions, which is why supplying the full taxonomy, rather than the broad labels alone, changes model behaviour so sharply.

B Error Analysis Examples

Three error types dominate. (i) *Cross-family topic leaks*: a description of a religious book (يهدف هذا الكتاب إلى قراءة سيرة الرسول, id 10041) is `islamic_book_description` (Informative), which our base and final systems both get right but which every broad-only verifier calls Religious; scripture leaks the other way (هؤلاء امراء بني عيسو, Genesis 36, id 10798),

where the base says `history_encyclopedia` and consensus corrects it to `bible`. (ii) *Attractor over-prediction*: long prose absorbed into `egyptian_song_lyrics`. (iii) *Ceiling classes*: for each of the five song-lyric classes, 87–99% of the texts we assign to it contain no marker from that dialect’s cue list (`src/dialect_markers.py` ships the lists and reproduces the table; the rate is a property of the text, so it is measurable without gold).

C Negative Results

A *direct* six-way broad classifier regressed to 0.6206, confirming that deriving broad from the specific prediction beats classifying it directly; scoring all 74 candidates without retrieval pruning fell to 0.6517. For correction, plain self-consistency (0.7134), entropic optimal transport (0.7115), uniform-prior calibration (0.7124) and an unrestricted chain-of-thought pass (0.7131) all regressed below the 0.7139 base, whereas full-context consensus improved it. Cross-lingual rerankers failed (Aarab, 2026), regex rules added nothing the judge lacked, and the off-the-shelf dialect classifiers we tried did not expose the Iraqi/Sudanese distinction that NADI-style country-level tasks define (Abdul-Mageed et al., 2021).

D Full-Context vs. Broad-Only Prompting

Both consensus models were run over the same verification pool ($n = 2,635$) once with only the 6 broad definitions and once with all 74 specific definitions. Because the full-taxonomy prompt also carries an explicit type-vs-topic instruction, we added a third arm, broad definitions only *plus* that instruction, to separate the two factors. On the 1,403 book-description instances GPT-5.6 calls 394 of them Religious with neither, 233 with the instruction alone and 119 with the full taxonomy; `src/trap_table.py` rebuilds these counts from cached verifier outputs with no API access.

Table 3 reports how often GPT-5.6’s broad verdict still agrees with the base system’s family, one model across the two original prompts. Broad-only prompting collapses on families whose members are *about* another family’s subject matter (Learning, Informative), which is exactly the topic trap; the full taxonomy repairs it. Requiring both veri-

fiers to agree, joint Religious assignments on the same 1,403 instances fall from 235 under broad-only prompting to 32 under the full taxonomy, an 86% reduction.

Base family	n	broad-only	full-taxonomy
Informative	1682	49%	78%
Learning	305	18%	74%
Interactive	153	46%	72%
Legal	281	53%	56%
Creative	151	50%	48%
Religious	63	41%	48%

Table 3: Share of instances where GPT-5.6’s broad verdict matches the base family, by prompt context. This measures *agreement with the base system*, not accuracy: the hidden test has no released gold, so we can show that full context changes verifier behaviour and that the change is consistent with the taxonomy’s own conventions, but we cannot certify it as an accuracy gain. The dev contrast in §6.1 is the gold-anchored counterpart.

E Attractor Drain

The drain is deterministic: it fires when the base label is an attractor class and the chain-of-thought pass names a different sibling inside the same broad family (Table 4). No score threshold or entropy criterion is used, and broad is held fixed by construction.

Class	base	moved
<code>literature_fiction_bk_desc.</code>	598	208 (35%)
<code>egyptian_song_lyrics</code>	979	212 (22%)
<code>msa_poetry</code>	608	52 (9%)
<code>culture_encyclopedia</code>	455	41 (9%)
<code>history_encyclopedia</code>	632	41 (6%)

Table 4: Attractor classes and how many predictions the drain reassigns to within-family siblings when applied to the base predictions alone. In the submitted pipeline consensus is applied first and the drain fires on 383 of these instances.

F Calibration

The correction subtracts a scaled per-genre mean from each candidate’s score, $s'(y) = s(y) - \alpha \bar{s}(y)$, where $\bar{s}(y)$ is the mean score genre y received over the whole test set and $\alpha=0.5$. It damps genres the judge rates generously everywhere, which is what starves rare siblings. We started from $\alpha=0.75$, chosen on the released training labels; it over-flattened once the label space grew from 7 to 74 classes. The correction is applied before the argmax and never

changes the broad family. Two alternatives we tried instead of it, uniform-prior calibration and entropic optimal transport, both scored below the base (Appendix C).

G Hyperparameters and Prompts

Retriever: Qwen3-Embedding-8B loaded in 4-bit NF4 with fp16 compute (the configuration the submitted run used; only the cosine ranking is consumed and NF4 preserves embedding direction), 4096-dim, L2-normalised, `max_seq_length` 512, the top 20 specific genres cached per text; the judge scores every sibling in the four small families (Creative 7, Religious 6, Interactive 4, Legal 3) and the top 6 retrieved in-family candidates in the two large ones. Judge: `gemma-4-31b` via vLLM, setwise 0–100 scoring in a single pass, temperature 0, seed 42, 3 retries with JSON-regex fallback, marginal correction $\alpha=0.5$. Consensus verifiers: GPT-5.6 Luna (`reasoning_effort=low`, ≤ 500 completion tokens) and Gemini-3.6 Flash (temperature 0, ≤ 400 output tokens), each given the full 74-definition taxonomy (2,884 tokens under `o200k_base`, giving a mean prompt of 3,034 tokens and a maximum of 5,185) in the system prompt, and the text truncated to 1500 characters. The judge prompt asks for a JSON array of integer scores over the candidate definitions; the consensus prompt asks for a single exact `specific_genre` identifier and states that a factual description *of* a book is a `*_book_description`, not the book’s subject genre.

Cost. At list prices the consensus stage runs about US\$1.05–1.48 per 1,000 instances, giving the \$50 total quoted in §5.

Throughput. A consensus call takes 2.0–2.8 s. At the released default of 20 concurrent workers the 8,041-instance Gemini pass took 19 min of wall clock; the full GPT pass over 27,972 instances was run at higher concurrency.

H Official Submission Record

Every hidden-test number we report is an official evaluation of a submitted file (Table 5), so the progression is traceable to scored submissions rather than to local approximations.

Two readings this record supports. Submission 867465 isolates the surface rules on top of 867323: they rewrite 32 labels (23 quran, 9

ID	System	Hier. F1
865908	genre-general zero-shot judge	0.6067
866538	+ embedding retrieval gate	0.6812
867323	family-restricted, derived broad	0.6882
867376	direct six-way broad judge	0.6206
867451	unpruned 74-way scoring	0.6517
867465	+ surface rules only	0.6881
870640	base system	0.7139
870841	uniform-prior calibration	0.7124
870979	self-consistency ($K=3$)	0.7134
870981	entropic optimal transport	0.7115
871127	chain-of-thought applied wholesale	0.7131
871609	+ 2-model consensus (pool)	0.7224
872511	+ safe broad expansion	0.7256
872515	+ full consensus + drain	0.7352

Table 5: Official submissions behind Table 1 and Appendix C. Submission 872515 is the final system: 27,972 predictions over 27,972 gold instances, none missing. Submissions were made sequentially over four days, so the progression is a record of scored configurations rather than a single controlled sweep.

hadith) and leave the score at 0.6881 against 0.6882. The rules are precise but score-neutral at that point, so the 0.6882→0.7139 gain belongs to the judge and calibration changes, not to them. They were kept for a different job: in the final pipeline they fire on 1,087 instances and agree with the base label on every one, so their whole net effect is to restore 75 labels that consensus or the drain had moved away (56 `youtube_comments`, 19 quran). They guard the correction stages rather than adding score. Submission 871127 applies the chain-of-thought pass to every instance and scores 0.7131, below the 0.7139 base; the same pass restricted to attractor classes is what the drain uses. Chain-of-thought helps here only where it is aimed.

Gated broad moves. Of the 3,300 instances where both verifiers agree against the base, 2,122 stay inside the base family and are applied as refinements. Of the 1,178 that would cross families, 460 fall in a read-verified direction and are applied: Informative→Interactive 195, Legal→Informative 80, Creative→Religious 58, Informative→Religious 58, Creative→Informative 44, Interactive→Religious 20, Legal→Interactive 5. The remaining 718 are rejected, the largest groups being Informative→Learning (219), Creative→Interactive (122) and Learning→Informative (99).

Preserved rule defects. Three regexes in `src/rules.py` do not match their intent: the

Qur'anic class also admits U+0671, the opinion class lets a bare variation selector match, and one character class contains a literal |. They ran as written when the submission was produced, so we keep them verbatim and document them in the code; correcting all three changes 22 of the 27,972 final predictions.

Orthographic signals. Share of each group carrying heavy diacritisation (a mark on over 10% of its Arabic characters) or at least one emoji, over the final predictions: quran 77%/0% ($n=691$), poetry 43%/0% ($n=1,606$), hadith 1%/1% ($n=647$), book descriptions 1%/0% ($n=3,916$), Interactive 0%/47% ($n=1,103$). Both are properties of the text, so no gold is needed; `src/orthographic_signals.py` reproduces the table.

Software. torch ≥ 2.1 , transformers ≥ 4.44 (<https://github.com/huggingface/transformers>), sentence-transformers ≥ 3.0 (<https://github.com/UKPLab/sentence-transformers>), bitsandbytes ≥ 0.43 (<https://github.com/bitsandbytes-foundation/bitsandbytes>), vLLM (<https://github.com/vllm-project/vllm>), openai ≥ 1.99 (<https://github.com/openai/openai-python>) and google-genai ≥ 0.3 (<https://github.com/googleapis/python-genai>). Model weights: Qwen3-Embedding-8B (<https://huggingface.co/Qwen/Qwen3-Embedding-8B>).